

Inigo Quilez :: articles :: 2D distance and gradient functions - 2019



Intro

We know that proper 2D and 3D SDFs have a gradient of length 1.0 everywhere, since SDFs measure distances, and the rate of change of distance with respect to distance will always be the identity. However, knowing the exact direction of the gradient is useful for computer graphics (for lighting, aligning objects to surfaces, etc etc). Of course, one can use **central differences to compute gradients**, but that has a high computational cost and can get tricky to tweak. Automatic differentiation works well with SDFs of course, but comes with a cost.

In this page you'll find a list of primitives and operators and their analytic gradients that I developed around 2020, where lots of terms are reused between the SDF computation and its gradient computation, making them very cheap. The list is incomplete, so feel free to carry on all primitives and operations that I haven't defined with automatic differentiation/duals. Also, this list just contains **2D SDFs**, but extending them to 3D through **revolution or extrusion** is pretty easy.

Lastly, note that all the primitives listed here come with a link to a realtime online demo in Shadertoy. In fact, this public playlist contains all the Shadertoy examples: <https://www.shadertoy.com/playlist/M3dSRf>, so don't miss that out.

Primitives

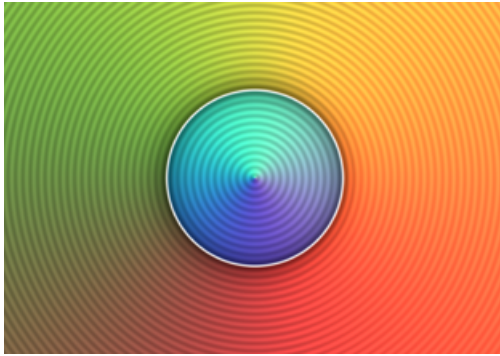
All primitives are centered at the origin; you will have to transform the point to get arbitrarily rotated, translated and scaled objects, and transform the gradient back accordingly (remember not to use non uniform scales since such transforms do not preserve distances). All SDFs below return the actual distance in the $.x$ component, and its partial derivatives with respect to x and y in the $.y$ and $.z$ components. In other words,

$$.x = f(p)$$

$$.y = \partial f(p) / \partial x$$

$$.z = \partial f(p) / \partial y$$

$.yz = \nabla f(p)$ with $\|\nabla f(p)\| = 1$



Circle

<https://www.shadertoy.com/view/WltSDj>

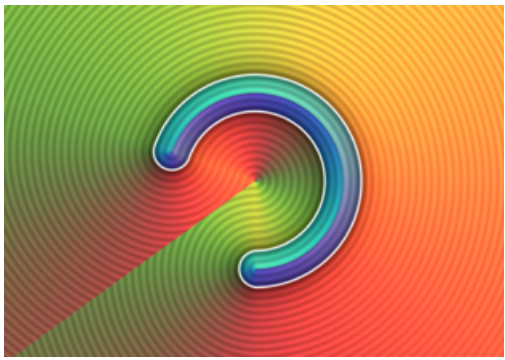
```
vec3 sdgCircle( in vec2 p, in float r )
{
    float d = length(p);
    return vec3( d-r, p/d );
}
```



Pie

<https://www.shadertoy.com/view/3tGXRc>

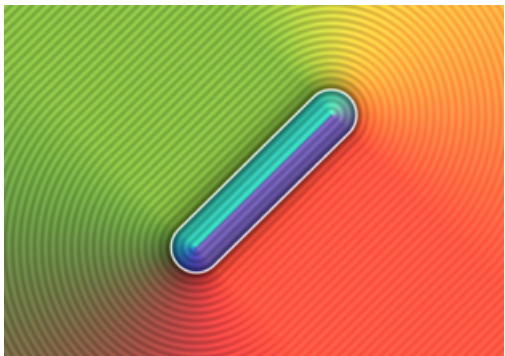
```
// sc = sin/cos of aperture
vec3 sdgPie( in vec2 p, in vec2 sc, in float r )
{
    float s = sign(p.x); p.x = abs(p.x);
    float l = length(p);
    float n = 1 - r;
    vec2 q = p - sc*clamp(dot(p,sc),0.0,r);
    float m = length(q) * sign(sc.y*p.x-sc.x*p.y);
    vec3 res = (n>m) ? vec3(n,p/l) : vec3(m,q/m);
    return vec3(res.x,s*res.y,res.z);
}
```



Arc

<https://www.shadertoy.com/view/WtGXRc>

```
// sc = sin/cos of aperture
vec3 sdgArc( in vec2 p, in vec2 sc, in float ra, in float rb )
{
    vec2 q = p;
    float s = sign(p.x); p.x = abs(p.x);
    if( sc.y*p.x > sc.x*p.y )
    {
        vec2 w = p - ra*sc;
        float d = length(w);
        return vec3( d-rb, vec2(s*w.x,w.y)/d );
    }
    else
    {
        float l = length(q);
        float w = l - ra;
        return vec3( abs(w)-rb, sign(w)*q/l );
    }
}
```



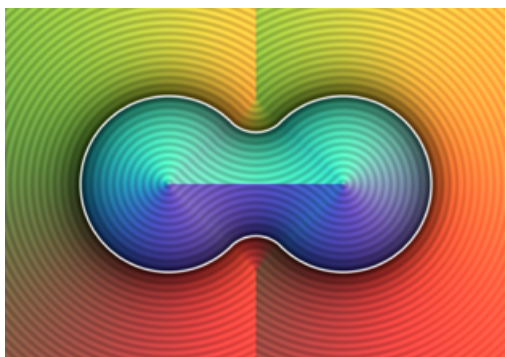
Segment

<https://www.shadertoy.com/view/WtdSDj>

```

vec3 sdgSegment( in vec2 p, in vec2 a, in vec2 b, in float r )
{
    vec2 ba = b-a, pa = p-a;
    float h = clamp( dot(pa,ba)/dot(ba,ba), 0.0, 1.0 );
    vec2 q = pa-h*ba;
    float d = length(q);
    return vec3(d-r,q/d);
}

```



Vesica

<https://www.shadertoy.com/view/3IGXRc>

```

vec3 sdgVesica( vec2 p, float r, float d )
{
    vec2 s = sign(p); p = abs(p);
    float b = sqrt(r*r-d*d);
    if( (p.y-b)*d>p.x*b )
    {
        vec2 q = vec2(p.x,p.y-b);
        float l = length(q)*sign(d);
        return vec3( l, s*q/l );
    }
    else
    {
        vec2 q = vec2(p.x+d,p.y);
        float l = length(q);
        return vec3( l-r, s*q/l );
    }
}

```



Box

<https://www.shadertoy.com/view/wlcXD2>

```

vec3 sdgBox( in vec2 p, in vec2 b )
{
    vec2 w = abs(p)-b;
    vec2 s = vec2(p.x<0.0?-1:1,p.y<0.0?-1:1);
    float g = max(w.x,w.y);
    vec2 q = max(w,0.0);
    float l = length(q);
    return vec3( (g>0.0)?1 :g,
                s*((g>0.0)?q/l:((w.x>w.y)?vec2(1,0):vec2(0,1)))));
}

```



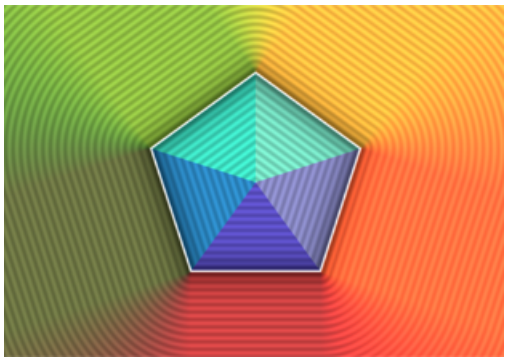
Cross

<https://www.shadertoy.com/view/WtdXWj>

```

vec3 sdgCross( in vec2 p, in vec2 b )
{
    vec2 s = sign(p); p = abs(p);
    vec2 q = ((p.y>p.x)?p.yx:p.xy) - b;
    float h = max( q.x, q.y );
    vec2 o = max( (h<0.0)?vec2(b.y-b.x,0.0)-q:q, 0.0 );
    float l = length(o);
    vec3 r = (h<0.0 && -q.x<1)?vec3(-q.x,1.0,0.0):vec3(1,o/l);
    return vec3( sign(h)*r.x, s*((p.y>p.x)?r.zy:r.yz) );
}

```



Pentagon

<https://www.shadertoy.com/view/3lySRc>

```
vec3 sdgPentagon( in vec2 p, in float r )
{
    const vec3 m = vec3(0.80901699,0.58778525,0.72654253);
    const vec2 n = vec2(m.x*m.x-m.y*m.y,2.0*m.x*m.y );
    float s = sign(p.x);
    p.x = abs(p.x);
    float w1 = p.x*m.x + p.y*m.y;
    float w2 = p.x*n.x - p.y*n.y;
    p -= 2.0*max(w1,0.0)*vec2(m.x,m.y);
    p -= 2.0*min(w2,0.0)*vec2(m.x,-m.y);
    p -= vec2(clamp(p.x,-r*m.z,r*m.z),-r);
    float d = length(p)*sign(-p.y);
    vec2 g = (w2<0.0)?mat2x2(-m.x,m.y,-m.y,-m.x)*p:
              (w1>0.0)?mat2x2(-n.x,-n.y,-n.y,n.x)*p:
              p;
    g.x *= s;
    return vec3(d, g/d );
}
```



Hexagon

<https://www.shadertoy.com/view/WtySRc>

```

vec3 sdgHexagon( in vec2 p, in float r )
{
    const vec3 k = vec3(-0.866025404,0.5,0.577350269);
    vec2 s = sign(p); p = abs(p);
    float w = dot(k.xy,p);
    p -= 2.0*min(w,0.0)*k.xy;
    p -= vec2(clamp(p.x, -k.z*r, k.z*r), r);
    float d = length(p)*sign(p.y);
    vec2 g = (w<0.0) ? mat2(-k.y,-k.x,-k.x,k.y)*p : p;
    return vec3( d, s*g/d );
}

```



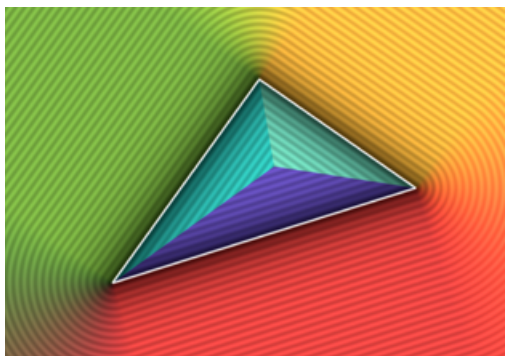
Isosceles Triangle

<https://www.shadertoy.com/view/3dyfDd>

```

vec3 sdgTriangleIsosceles( in vec2 p, in vec2 q )
{
    float w = sign(p.x);
    p.x = abs(p.x);
    vec2 a = p - q*clamp( dot(p,q)/dot(q,q), 0.0, 1.0 );
    vec2 b = p - q*vec2( clamp( p.x/q.x, 0.0, 1.0 ), 1.0 );
    float k = sign(q.y);
    float l1 = dot(a,a);
    float l2 = dot(b,b);
    float d = sqrt((l1<l2)?l1:l2);
    vec2 g = (l1<l2)? a: b;
    float s = max( k*(p.x*q.y-p.y*q.x),k*(p.y-q.y) );
    return vec3(d,vec2(w*g.x,g.y)/d)*sign(s);
}

```



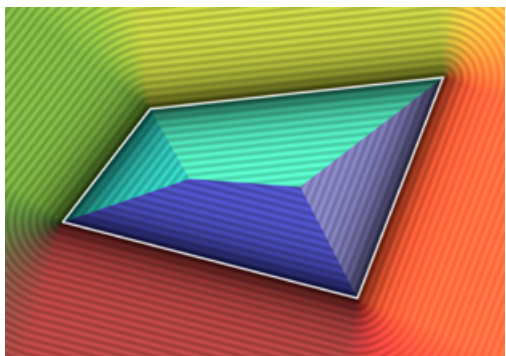
Triangle

<https://www.shadertoy.com/view/tlVyWh>

```
float cro( in vec2 a, in vec2 b ) { return a.x*b.y - a.y*b.x; }
vec3 sdgTriangle( in vec2 p, in vec2 v[3] )
{
    float gs = cro(v[0]-v[2],v[1]-v[0]);
    vec4 res;

    {
        vec2 e = v[1]-v[0], w = p-v[0];
        vec2 q = w-e*clamp(dot(w,e)/dot(e,e),0.0,1.0);
        float d = dot(q,q), s = gs*cro(w,e);
        res = vec4(d,q,s);
    } {
        vec2 e = v[2]-v[1], w = p-v[1];
        vec2 q = w-e*clamp(dot(w,e)/dot(e,e),0.0,1.0);
        float d = dot(q,q), s = gs*cro(w,e);
        res = vec4( (d<res.x) ? vec3(d,q) : res.xyz,
                    (s>res.w) ?      s      : res.w );
    } {
        vec2 e = v[0]-v[2], w = p-v[2];
        vec2 q = w-e*clamp(dot(w,e)/dot(e,e),0.0,1.0);
        float d = dot(q,q), s = gs*cro(w,e);
        res = vec4( (d<res.x) ? vec3(d,q) : res.xyz,
                    (s>res.w) ?      s      : res.w );
    }

    float d = sqrt(res.x)*sign(res.w);
    return vec3(d,res.yz/d);
}
```



Quad

<https://www.shadertoy.com/view/WtVcD1>

```
float cro( in vec2 a, in vec2 b ) { return a.x*b.y - a.y*b.x; }
vec3 sdgQuad( in vec2 p, in vec2 v[4] )
{
    float gs = cro(v[0]-v[3],v[1]-v[0]);
    vec4 res;

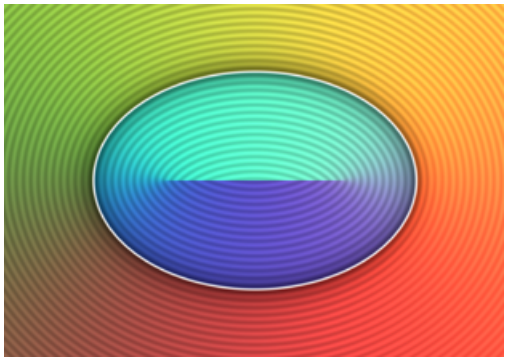
    {
        vec2 e = v[1]-v[0], w = p-v[0];
        vec2 q = w-e*clamp(dot(w,e)/dot(e,e),0.0,1.0);
        float d = dot(q,q), s = gs*cro(w,e);
        res = vec4(d,q,s);
    } {
        vec2 e = v[2]-v[1], w = p-v[1];
        vec2 q = w-e*clamp(dot(w,e)/dot(e,e),0.0,1.0);
        float d = dot(q,q), s = gs*cro(w,e);
        res = vec4( (d<res.x) ? vec3(d,q) : res.xyz,
                    (s>res.w) ?      s      : res.w );
    } {
        vec2 e = v[3]-v[2], w = p-v[2];
        vec2 q = w-e*clamp(dot(w,e)/dot(e,e),0.0,1.0);
        float d = dot(q,q), s = gs*cro(w,e);
        res = vec4( (d<res.x) ? vec3(d,q) : res.xyz,
                    (s>res.w) ?      s      : res.w );
    } {
        vec2 e = v[0]-v[3], w = p-v[3];
        vec2 q = w-e*clamp(dot(w,e)/dot(e,e),0.0,1.0);
        float d = dot(q,q), s = gs*cro(w,e);
        res = vec4( (d<res.x) ? vec3(d,q) : res.xyz,
                    (s>res.w) ?      s      : res.w );
    }

    float d = sqrt(res.x)*sign(res.w);
```

```

    return vec3(d,res.yz/d);
}

```



Ellipse

<https://www.shadertoy.com/view/3lcfR8>

```

vec3 sdgEllipse( vec2 p, in vec2 ab )
{
    vec2 sp = sign(p); p = abs( p );

    bool s = dot(p/ab,p/ab)>1.0;
    float w = atan(p.y*ab.x, p.x*ab.y);
    if(!s) w=(ab.x*(p.x-ab.x)<ab.y*(p.y-ab.y))? 1.570796327 : 0.0;

    for( int i=0; i<4; i++ )
    {
        vec2 cs = vec2(cos(w),sin(w));
        vec2 u = ab*vec2( cs.x,cs.y);
        vec2 v = ab*vec2(-cs.y,cs.x);
        w = w + dot(p-u,v)/(dot(p-u,u)+dot(v,v));
    }
    vec2 q = ab*vec2(cos(w),sin(w));

    float d = length(p-q);
    return vec3( d, sp*(p-q)/d ) * (s?1.0:-1.0);
}

```



Moon

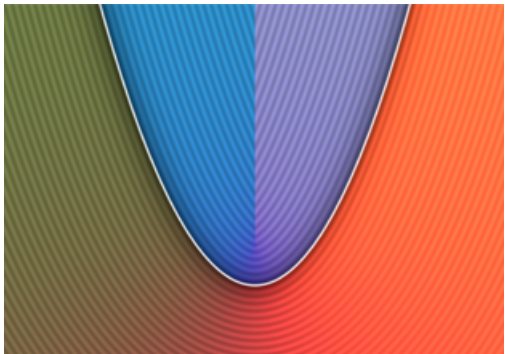
<https://www.shadertoy.com/view/ddX3WH>

```
vec3 sdMoon(vec2 p, float d, float ra, float rb )
{
    float s = sign(p.y);
    p.y = abs(p.y);

    float a = (ra*ra - rb*rb + d*d)/(2.0*d);
    float b = sqrt(max(ra*ra-a*a,0.0));
    if( d*(p.x*b-p.y*a) > d*d*max(b-p.y,0.0) )
    {
        vec2 w = p-vec2(a,b); float d = length(w); w.y *= s;
        return vec3(d,w/d);
    }

    vec2 w1 = p;
    vec2 w2 = p-vec2(d,0);
    float l1 = length(w1); float d1 = l1-ra; w1.y *= s;
    float l2 = length(w2); float d2 = rb-l2; w2.y *= s;

    return (d1>d2) ? vec3(d1,w1/l1) : vec3(d2,-w2/l2);
}
```



Parabola

<https://www.shadertoy.com/view/mdX3WH>

```
vec3 sdgParabola( in vec2 pos, in float k )
{
    float s = sign(pos.x);
    pos.x = abs(pos.x);

    float ik = 1.0/k;
    float p = ik*(pos.y - 0.5*ik)/3.0;
    float q = 0.25*ik*ik*pos.x;
    float h = q*q - p*p*p;
    float r = sqrt(abs(h));

    float x = (h>0.0) ?
        pow(q+r,1.0/3.0) - pow(abs(q-r),1.0/3.0)*sign(r-q) :
        2.0*cos(atan(r,q)/3.0)*sqrt(p);

    float z = sign(pos.x-x);
    vec2 w = pos-vec2(x,k*x*x); float l = length(w); w.x*=s;
    return z*vec3(l, w/l );
}
```



Trapezoid

<https://www.shadertoy.com/view/ddt3Rs>

```
vec3 sdgTrapezoid( vec2 p, float ra, float rb, float he, out vec2 oc )
{
    float sx = (p.x<0.0)?-1.0:1.0;
    float sy = (p.y<0.0)?-1.0:1.0;

    p.x = abs(p.x);

    vec4 re;
    {
        float h = min(p.x, (p.y<0.0)?ra:rb);
```

```

    vec2 c = vec2(h, sy*he);
    vec2 q = p - c;
    float d = dot(q,q);
    float s = abs(p.y) - he;
    re = vec4(d,q,s);
    oc = c;
}
{
    vec2 k = vec2(rb-ra, 2.0*he);
    vec2 w = p - vec2(ra, -he);
    float h = clamp(dot(w,k)/dot(k,k), 0.0, 1.0);
    vec2 c = vec2(ra, -he) + h*k;
    vec2 q = p - c;
    float d = dot(q,q);
    float s = w.x*k.y - w.y*k.x;
    if( d<re.x ) { oc = c; re.xyz = vec3(d,q); }
    if( s>re.w ) { re.w = s; }
}

float d = sqrt(re.x)*sign(re.w);
re.y *= sx;
oc.x *= sx;

return vec3(d, re.yz/d);
}

```



Heart

<https://www.shadertoy.com/view/DldXRf>

```

vec3 sdgHeart( in vec2 p )
{
    float sx = (p.x<0.0)?-1.0:1.0;
    p.x = abs(p.x);

    if( p.y+p.x>1.0 )
    {
        const float r = sqrt(2.0)/4.0;
        vec2 q0 = p - vec2(0.25, 0.75);
        float l = length(q0);
        vec3 d = vec3(l-r, q0/l);
        d.y *= sx;
        return d;
    }
    else
    {
        vec2 q1 = p - vec2(0.0, 1.0);
        vec2 q2 = p - 0.5*max(p.x+p.y, 0.0);
        vec3 d1 = vec3(dot(q1,q1), q1);

```

```

        vec3 d2 = vec3(dot(q2,q2),q2);
        vec3 d = (d1.x<d2.x) ? d1: d2;
        d.x = sqrt(d.x);
        d.yz /= d.x;
        d *= (p.x>p.y)?1.0:-1.0;
        d.y *= sx;
        return d;
    }
}

```

Operators/modifiers

As said, once the gradients of the basic primitives are hardcoded, automatic differentiation can take over for most of the remaining composition work. However, if you really want to make it all crystal clear and remove all opaque aspects of your code, you might want to implement analytic gradient computation for the composition operators too. Here go a few:

Rounding

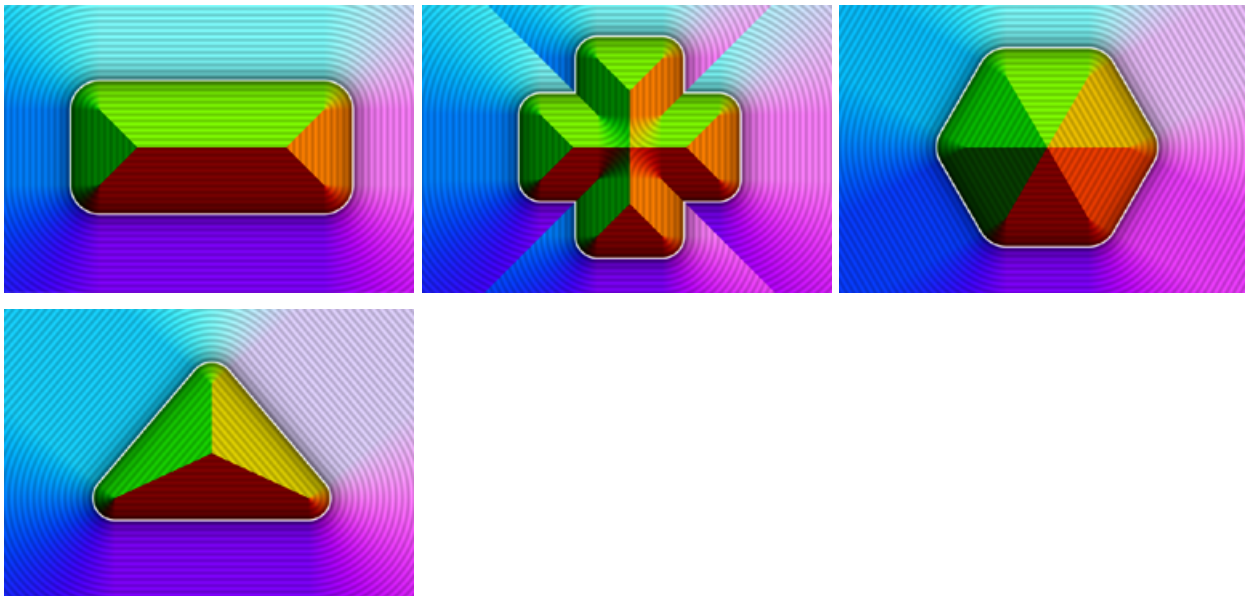
As we know, inflating any shape into a rounded self is pretty easy with the single subtraction of a quantity r . If such r is a constant, the derivatives of the SDF won't change, so we don't really need to do anything in order to continue getting correct gradients.

```

vec3 sdgRound( in vec2 p, in float r )
{
    vec3 dis_gra = sdgShape(p);
    return vec3( dis_gra.x - r, dis_gra.yz );
}

```

These are a few examples:

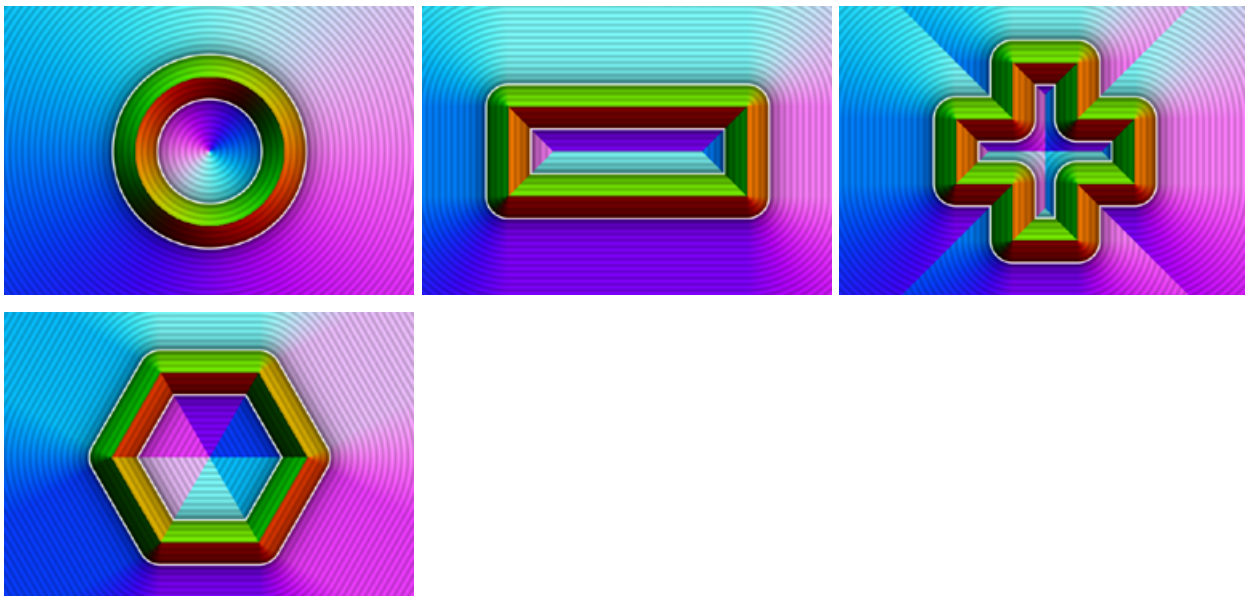


Onion - annular shapes

Similarly, shapes can be made annular (like a ring or the layers of an onion) by taking their absolute value and then subtracting a constant from their field. This operation is also very easy to carry over for the gradients:

```
vec3 sdgOnion( in vec2 p, in float r )
{
    vec3 dis_gra = sdgShape(p);
    return vec3( abs(dis_gra.x) - r, sign(dis_gra.x)*dis_gra.yz );
}
```

These are a few examples:



Minimum and Smooth-minimum

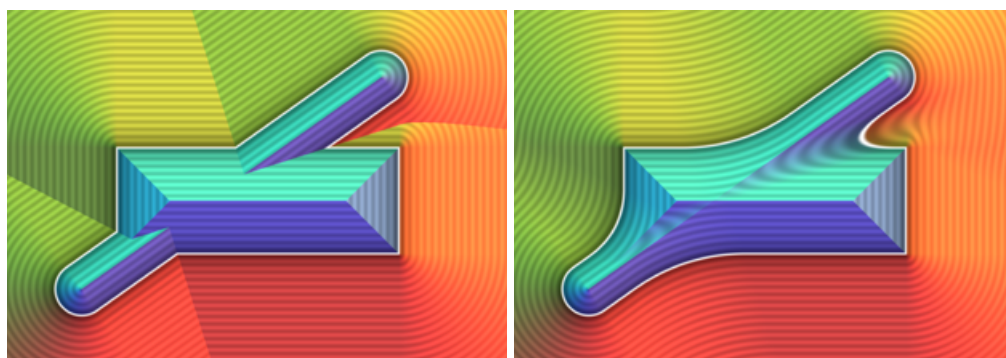
<https://www.shadertoy.com/view/tdGBDt>

One way to combine shapes is to take the minimum of two SDFs. Naturally, the same conditional branching that picks the smallest of the SDFs should be used to select and propagate the gradient. However, when a **smooth-minimum** is used to smoothly connect shapes, the derivative selection is more involved. Luckily, using polynomial smooth-minimums have simple derivatives and accept an analytical propagation. Of course, bear in mind that the smooth-minimum does **not** respect the constraints of having unit length everywhere, so its output is only a true SDF (or almost) far away from the surface of the resulting shape. You can learn more about this and to derive the formula below in the **Smooth-minimum** article, but I leave the code here too for the analytic derivatives/gradient of a quadratic smooth-minimum for convenience:

```
vec3 min( in vec3 a, in vec3 b )
{
    return (a.x<b.x)?a:b;
}

vec3 smin( in vec3 a, in vec3 b, in float k )
{
    k *= 4.0;
    float h = max(k-abs(a.x-b.x),0.0);
    float m = 0.25*h*h/k;
    float n = 0.50* h/k;
    return vec3( min(a.x, b.x) - m,
                 mix(a.yz, b.yz, (a.x<b.x)?n:1.0-n) );
}
```

Here's on the left the minimum, and on the right the smooth-minimum:



inigo quilez - learning computer graphics since 1994